Foundations of AI & ML **Fall 2026**
Professor Amy Greenwald[1] **Supervised Learning Unit**

Feed Forward Neural Networks

In this lecture, we present the classic backpropagation algorithm for training an artificial neural network. This algorithm learns the network's weights and biases with the goal of minimizing an appropriate loss function. The learning algorithm of choice is gradient descent, which means we must compute the partial derivative of the loss with respect to every parameter in the network. Backpropagation is an efficient way to compute all of these derivatives at once. It has two phases: a **forward pass**, which computes the network's output and stores the intermediate quantities along the way, and a **backward pass**, which reuses those quantities to propagate derivatives from the output all the way back to the input.

1 Notation

Recall Figure 1 from last lecture, which depicts a neural network with two hidden layers.

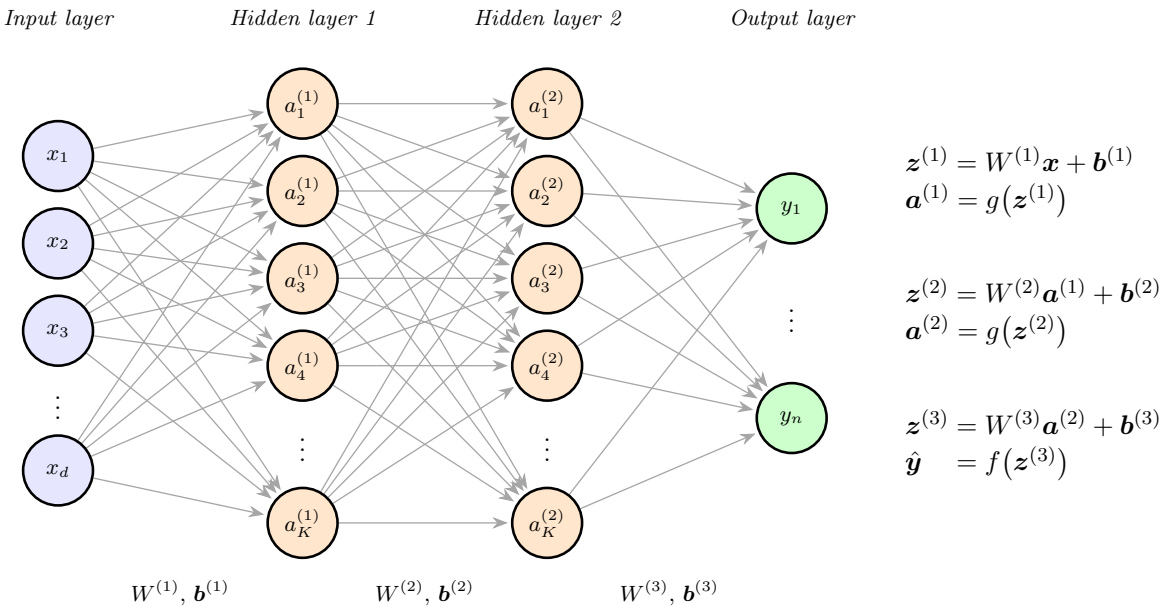


Figure 1: A feed-forward artificial neural network with two hidden layers learning from examples of the form $(\boldsymbol{x}, \boldsymbol{y}) \in \mathbb{R}^d \times \mathbb{R}^n$. The output activation f is usually the identity in a regression problem, the sigmoid function in binary classification, and softmax in multiclassification. The activation functions g applied at each hidden layer are often nonlinear, differentiable functions, like tanh or (a smooth version of) ReLU.

More generally, assume a network with L layers, numbered $l \in \{0, 1, \ldots, L\}$, where layer 0 is the input and layer L is the output. Layer l is of size $k^{(l)}$. The input layer is of size $k^{(0)} = d$, one per feature x_i, and the output layer is of size $k^{(L)}$, e.g., n in a multiclassification problem with n classes. Furthermore, assume all hidden layers have K neural units, i.e., $k^{(l)} = K$, for all $l \in \{1, \ldots, L-1\}$.

[1]These notes were compiled in conjunction with Professor Eric Ewing.

For each layer $l \geq 1$, we write:

- $w_{ij}^{(l)}$ for the **weight** on the connection *from* unit j in layer $l-1$ *to* unit i in layer l
- $b_i^{(l)}$ for unit i's **bias** in layer l
- $z_i^{(l)}$ for unit i's **pre-activation** in layer l
- $a_i^{(l)}$ for unit i's **activation** (output) in layer l
- $g_i^{(l)}$ for unit i's **activation function** in layer l

At layer l, the pre-activations, activations and biases form column vectors $\boldsymbol{z}^{(l)}$, $\boldsymbol{a}^{(l)}$ and $\boldsymbol{b}^{(l)}$ of length $k^{(l)}$, one per unit. The weights form a $k^{(l)} \times k^{(l-1)}$ matrix $W^{(l)}$, where by convention, a row $w_i^{(l)}$ stores the weights on the links from all the units in the previous layer into unit i, while column $W_{:j}^{(l)}$ stores the weights on the links from unit j into all the units in the next layer.

We describe the forward and backward passes for a single training example $(\boldsymbol{x}, \boldsymbol{y})$.

2 The Forward Pass

The forward pass starts by copying the input vector into layer 0, so that

$$\boldsymbol{a}^{(0)} = \boldsymbol{x} \in \mathbb{R}^d \tag{1}$$

At each subsequent layer, the same two steps are applied to all its units in parallel: a weighted sum followed by a nonlinearity. For $l = 1, \ldots, L$,

$$\boldsymbol{z}^{(l)} = W^{(l)} \boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)} \tag{2}$$

$$\boldsymbol{a}^{(l)} = g^{(l)} \left(\boldsymbol{z}^{(l)} \right) \tag{3}$$

where $g^{(l)}$ is applied elementwise.

RemarK Equation 2 is a compact way of writing one weighted sum per unit. Its ith entry is given by:

$$z_i^{(l)} = \sum_{j=1}^{k^{(l-1)}} w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)} \tag{4}$$

Intuitively, unit i at layer l is fed by every unit at layer $l-1$ and a bias. It sums the activations of all those preceding units, each weighted by the strength of its connection, and then adds its bias.

The whole forward pass is thus a loop of matrix-vector products and elementwise nonlinearities (see Table 1):

$$\boldsymbol{x} = \boldsymbol{a}^{(0)} \xrightarrow{W^{(1)}, \boldsymbol{b}^{(1)}} \boldsymbol{z}^{(1)} \xrightarrow{g} \boldsymbol{a}^{(1)} \xrightarrow{W^{(2)}, \boldsymbol{b}^{(2)}} \cdots \xrightarrow{W^{(L)}, \boldsymbol{b}^{(L)}} \boldsymbol{z}^{(L)} \xrightarrow{g} \boldsymbol{a}^{(L)} = \hat{\boldsymbol{y}}$$

The backward pass reads every $\boldsymbol{z}^{(l)}$ and $\boldsymbol{a}^{(l)}$, so they are stored (as memory allows) as they are computed.

$W^{(l)}$	$W^{(l)}\boldsymbol{a}^{(l-1)}$	$\boldsymbol{b}^{(l)}$	$\boldsymbol{z}^{(l)} = W^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)}$	$\boldsymbol{a}^{(l)} = g^{(l)}\left(\boldsymbol{z}^{(l)}\right)$
—	—	—	—	$\boldsymbol{a}^{(0)} = \boldsymbol{x} \in \mathbb{R}^d$
$W^{(1)} \in \mathbb{R}^{K \times d}$	$W^{(1)}\boldsymbol{a}^{(0)} \in \mathbb{R}^K$	$\boldsymbol{b}^{(1)} \in \mathbb{R}^K$	$\boldsymbol{z}^{(1)} \in \mathbb{R}^K$	$\boldsymbol{a}^{(1)} \in \mathbb{R}^K$
$W^{(2)} \in \mathbb{R}^{K \times K}$	$W^{(2)}\boldsymbol{a}^{(1)} \in \mathbb{R}^K$	$\boldsymbol{b}^{(2)} \in \mathbb{R}^K$	$\boldsymbol{z}^{(2)} \in \mathbb{R}^K$	$\boldsymbol{a}^{(2)} \in \mathbb{R}^K$
$W^{(3)} \in \mathbb{R}^{n \times K}$	$W^{(3)}\boldsymbol{a}^{(2)} \in \mathbb{R}^n$	$\boldsymbol{b}^{(3)} \in \mathbb{R}^n$	$\boldsymbol{z}^{(3)} \in \mathbb{R}^n$	$\boldsymbol{a}^{(3)} = \hat{\boldsymbol{y}} \in \mathbb{R}^n$

Table 1: The forward pass on the network in Figure 1, with dimensions made explicit.

3 The Backward Pass

Let $\mathcal{L}(\hat{\boldsymbol{y}}, \boldsymbol{y})$ denote the loss incurred when the network outputs $\hat{\boldsymbol{y}}$ on input $\boldsymbol{x}$ with label $\boldsymbol{y}$. Since $\hat{\boldsymbol{y}}$ is the output of the forward pass, the loss $\mathcal{L}$ is ultimately a function of the network's weights and biases $W^{(1)}$, $\boldsymbol{b}^{(1)}, \ldots, W^{(L)}$, $\boldsymbol{b}^{(L)}$. The goal of the backward pass is to compute, for all layers $l \in \{1, \ldots, L\}$,

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} \in \mathbb{R}^{k^{(l)} \times k^{(l-1)}} \qquad \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(l)}} \in \mathbb{R}^{k^{(l)}} \tag{5}$$

Observe that the derivative of the loss function $\mathcal{L}$ with respect to the weight matrix W has the same shape as W, and is defined entrywise: the ijth entry of $\partial\mathcal{L}/\partial W^{(l)}$ is $\partial\mathcal{L}/\partial w_{ij}^{(l)}$. Similarly, the derivative of $\mathcal{L}$ with respect to the vectors $\boldsymbol{a}, \boldsymbol{b}$, and $\boldsymbol{z}$ have the same shapes as $\boldsymbol{a}, \boldsymbol{b}$, and $\boldsymbol{z}$ and are also defined entrywise: e.g., the ith entry of $\partial\mathcal{L}/\partial\boldsymbol{b}^{(l)}$ is $\partial\mathcal{L}/\partial b_i^{(l)}$. This property makes it straightforward to apply gradient descent.

The central quantity of interest in the backward pass is the derivative of the loss with respect to each layer's pre-activations. This quantity is called the **error signal**. At layer l, it is defined as

$$\boldsymbol{\delta}^{(l)} \doteq \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}^{(l)}} \in \mathbb{R}^{k^{(l)}} \qquad \delta_i^{(l)} \doteq \frac{\partial \mathcal{L}}{\partial z_i^{(l)}} \in \mathbb{R} \tag{6}$$

Intuitively, $\delta_i^{(l)}$ measures how sensitive the loss is to a small change in unit i's pre-activation at layer l.

Backpropagation is a repeated application of the chain rule. Suppose a function F depends on a variable u through a chain of intermediate quantities, where each quantity is computed from the one before: i.e.,

$$u \longrightarrow v \longrightarrow w \longrightarrow F$$

By the **chain rule**,

$$\frac{\partial F}{\partial u} = \frac{\partial F}{\partial w}\frac{\partial w}{\partial v}\frac{\partial v}{\partial u} \tag{7}$$

Each factor in this expression is a **local derivative**: it measures how a quantity changes in response to a small change in the quantity immediately before it in the chain of computation.

3.1 Example: Regression with Squared Error

Consider a regression problem with a scalar label $y \in \mathbb{R}$. The output activation function in regression problems is typically the identity,[2] i.e., $g^{(L)}(z) = z$.

[2]Another viable choice might be ReLU or Softplus, for example, when the dependent variable is known to be non-negative (e.g., prices); or Softmax, when the dependent variable is a probability (e.g., the probability of defaulting on a loan).

In addition, since the label is a scalar value, the output layer of a neural network that models this problem is a single unit. The network's prediction on input $\boldsymbol{x} \in \mathbb{R}^d$ is therefore the scalar value

$$\hat{y} = a_1^{(L)} = z_1^{(L)}$$

We measure the error of this prediction as squared error:

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2 \tag{8}$$

In the remainder of this discussion, we drop the subscript 1 since there is only one output, as well as the superscript L, since it is clear from context.

With just one output unit, $W \in \mathbb{R}^{1 \times K}$ is a row vector and the bias b is a scalar, so that

$$z = \sum_{j=1}^{K} w_j a_j^{(L-1)} + b$$

For a weight w_j influences the loss along such a path, as does a bias b:

$$w_j \xrightarrow[\text{addition}]{\text{multiplication}} z \xrightarrow{g} a = \hat{y} \longrightarrow \mathcal{L} \qquad\qquad b \xrightarrow{\text{addition}} z \xrightarrow{g} a = \hat{y} \longrightarrow \mathcal{L}$$

Therefore, by the chain rule,

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial \mathcal{L}}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial w_j} \qquad\qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial b} \tag{9}$$

Each of the three local derivatives is straightforward:

$$\frac{\partial \mathcal{L}}{\partial a} = \hat{y} - y \qquad \text{by differentiating Equation 8}$$

$$\frac{\partial a}{\partial z} = \frac{\partial g(z)}{\partial z} = \frac{\partial z}{\partial z} = 1 \qquad \text{the derivative of the identity is 1}$$

$$\frac{\partial z}{\partial w_j} = a_j^{(L-1)} \qquad w_j \text{ only appears in the } j\text{th term of the summation}$$

Multiplying these local derivatives together yields

$$\frac{\partial \mathcal{L}}{\partial w_j} = (\hat{y} - y)\, a_j^{(L-1)} \tag{10}$$

The only factor that changes for a bias b is the last one, but $\partial z / \partial b = 1$, so

$$\frac{\partial \mathcal{L}}{\partial b} = \hat{y} - y \tag{11}$$

Observe that the first two factors in both calculations are the same. Together they form the error signal:

$$\delta = \frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a}\frac{\partial a}{\partial z} = \hat{y} - y \tag{12}$$

Equations 10 and 11 can thus be written as

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial \mathcal{L}}{\partial z}\frac{\partial z}{\partial w_j} = \delta\, a_j^{(L-1)} \qquad\qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial z}\frac{\partial z}{\partial b} = \delta$$

More generally, when accounting for all weights simultaneously,

$$\frac{\partial \mathcal{L}}{\partial W} = \delta \left(\boldsymbol{a}^{(L-1)}\right)^\top \in \mathbb{R}^{1\times K} \qquad\qquad \frac{\partial \mathcal{L}}{\partial b} = \delta \in \mathbb{R}$$

As promised, each gradient has the same shape as the parameter it differentiates.

3.2 Example: Binary Classification with Cross-Entropy Error

Now consider a binary classification problem with label $\{0, 1\}$. The output activation function in such problems is typically the sigmoid, i.e., $g^{(L)}(z) = \sigma(z) = \frac{1}{1+e^{-z}}$, which squashes the pre-activation into the interval $(0, 1)$, so that the output can be interpreted as the probability that the label is 1.

As in regression, the label is a scalar value, so the output layer of a neural network that models this problem is a single unit. The network's prediction on input $\boldsymbol{x} \in \mathbb{R}^d$ is therefore the scalar value

$$\hat{y} = a_1^{(L)} = \sigma\left(z_1^{(L)}\right) \in (0, 1)$$

We measure the error of this prediction as (binary) cross-entropy:

$$\mathcal{L}(\hat{y}, y) = -\left(y \log \hat{y} + (1-y)\log(1-\hat{y})\right) \tag{13}$$

When $y = 1$, only the first term survives, and the loss is $-\log \hat{y}$, which is small when $\hat{y}$ is close to 1 and grows without bound as $\hat{y} \to 0$. Symmetrically, when $y = 0$, the loss is $-\log(1-\hat{y})$.

As above, we drop the subscript 1 and the superscript L. With just one output unit, $W \in \mathbb{R}^{1\times K}$ is a row vector and the bias b is a scalar, so that

$$z = \sum_{j=1}^{K} w_j a_j^{(L-1)} + b$$

A weight w_j and a bias b influence the loss exactly as in regression; the only difference is that the activation function is now σ:

$$w_j \xrightarrow[\text{addition}]{\text{multiplication}} z \xrightarrow{\sigma} a = \hat{y} \longrightarrow \mathcal{L} \qquad\qquad b \xrightarrow{\text{addition}} z \xrightarrow{\sigma} a = \hat{y} \longrightarrow \mathcal{L}$$

Therefore, by the chain rule,

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial \mathcal{L}}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial w_j} \qquad\qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial b} \tag{14}$$

The last local derivative is the same as in regression, but the first two change:

$$\frac{\partial \mathcal{L}}{\partial a} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \qquad \text{by differentiating Equation 13}$$

$$\frac{\partial a}{\partial z} = \frac{\partial \sigma(z)}{\partial z} = \sigma(z)\big(1-\sigma(z)\big) = \hat{y}(1-\hat{y}) \qquad \text{the derivative of the sigmoid}$$

$$\frac{\partial z}{\partial w_j} = a_j^{(L-1)} \qquad w_j \text{ only appears in the } j\text{th term of the summation}$$

Multiplying these local derivatives together, the factor $\hat{y}(1-\hat{y})$ cancels, which yields

$$\frac{\partial \mathcal{L}}{\partial w_j} = (\hat{y} - y)\, a_j^{(L-1)} \tag{15}$$

Once again, the only factor that changes for the bias b is the last one, and $\partial z/\partial b = 1$, so

$$\frac{\partial \mathcal{L}}{\partial b} = \hat{y} - y \tag{16}$$

The first two factors in both calculations again form the error signal:

$$\delta = \frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a}\frac{\partial a}{\partial z} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) = \hat{y} - y \tag{17}$$

Equations 15 and 16 can thus be written exactly as before:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial \mathcal{L}}{\partial z}\frac{\partial z}{\partial w_j} = \delta\, a_j^{(L-1)} \qquad\qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial z}\frac{\partial z}{\partial b} = \delta$$

Remark Remarkably, the error signal $\delta = \hat{y} - y$ is *identical* to the one we found in regression, even though both the output activation and the loss changed. This is not an accident! Cross-entropy is paired with the sigmoid precisely because the $\hat{y}(1-\hat{y})$ in the denominator of $\partial\mathcal{L}/\partial a$ cancels the $\hat{y}(1-\hat{y})$ in $\partial a/\partial z$.

3.3 Gradient Descent

The identical error signals in both examples imply an identical gradient descent update for both. Specifically, the gradient with respect to w_j is the error at the output times the activation flowing along that connection. A gradient descent step with learning rate $\alpha > 0$ updates as follows:

$$\Delta w_j = -\alpha \frac{\partial \mathcal{L}}{\partial w_j} = \alpha\,(y - \hat{y})\, a_j^{(L-1)}$$

This rule should look familiar. Conceptually, it is identical to the perceptron learning rule. The only difference is that it replaces the input feature x_j with its generalization, namely the activation $a_j^{(L-1)}$.

Unlike for the perceptron, there is no need to consider four cases to confirm that this update moves in the right direction; just two suffice. The update to w_j changes the pre-activation by

$$\Delta z = \Delta w_j\, a_j^{(L-1)} = \alpha\,(y - \hat{y})\left(a_j^{(L-1)}\right)^2,$$

which has the same sign as $y - \hat{y}$, since $\left(a_j^{(L-1)}\right)^2 \geq 0$. In both examples, the prediction $\hat{y}$ is an increasing function of z: the identity in regression, and the sigmoid in binary classification. Hence, this update increases the prediction when it is too low, and decreases it when it is too high.